Prac1

```
import hashlib, datetime

class Block:
    def __init__(self, i, data, prev):
        self.index, self.data, self.previous_hash = i,
data, prev
        self.timestamp = str(datetime.datetime.now())
        self.hash = hashlib.sha256((str(i) +
self.timestamp + data + prev).encode()).hexdigest()

b = Block(1, "Genesis Block", "0")

print("Index:", b.index, "\nData:", b.data, "\nHash:",
b.hash)
```

Prac2

```
import hashlib, datetime

class Block:
    def __init__(self, i, tx, prev):
        self.index, self.transactions, self.previous_hash,
```

```
self.nonce = i, tx, prev, 0
        self.timestamp = str(datetime.datetime.now())
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        return hashlib.sha256((str(self.index) +
self.timestamp + str(self.transactions) +
self.previous_hash +
str(self.nonce)).encode()).hexdigest()

    def display(self):
        print(self.index, self.timestamp,
self.transactions, self.previous_hash, self.nonce,
self.hash)

b1 = Block(1, ["Alice pays Bob 10 BTC"], "0")
b1.display()

b2 = Block(2, ["Bob pays Charlie 5 BTC"], b1.hash)
b2.display()
```

Prac4

```
import java.security.*;
```

```
import java.util.*;

public class Merkle_tree {
    static String hash(String s) throws Exception {
        return HexFormat.of().formatHex(

MessageDigest.getInstance("SHA-256").digest(s.get
Bytes()));
    }

    public static void main(String[] args) throws
Exception {
        List<String> l = new
ArrayList<>(Arrays.asList("TX1", "TX2", "TX3", "TX4",
"TX5"));

        while (l.size() > 1) {
            List<String> n = new ArrayList<>();
            for (int i = 0; i < l.size(); i += 2)
                n.add(hash(l.get(i) + (i + 1 < l.size() ? l.get(i
+ 1) : l.get(i))));
            l = n;
        }

        System.out.println("Merkle Root: " + l.get(0));
```

```
    }
}
```

Prac5

```
const crypto = require("crypto");

class Wallet {
    constructor() {
        this.privateKey =
crypto.randomBytes(32).toString("hex");
        this.publicKey =
crypto.createHash("sha256").update(this.privateKey)
.digest("hex");
        this.address = this.publicKey.substring(0, 10);
        this.balance = 900000;
        this.transactions = [];
    }

    send(to, amount) {
        if (amount > this.balance) return false;
        this.balance -= amount;
        this.transactions.push({ to, amount });
```

```
        return true;
    }
}

let w = new Wallet();
console.log(w.address, w.balance, w.send("receiver",
900000), w.balance, w.transactions);
```

Prac6

```
import java.security.*;
import java.util.*;

class SimpleBlockchain {
    static String hash(String s) throws Exception {
        return HexFormat.of().formatHex(

MessageDigest.getInstance("SHA-256").digest(s.get
Bytes()));
    }

    public static void main(String[] a) throws
Exception {
        List<String> chain = new ArrayList<>();
```

```
        String prev = "0";

        for (int i = 0; i < 4; i++) {
            String data = "Transaction " + i;
            prev = hash(i + data + prev);
            chain.add(prev);
            System.out.println("Block " + i + ": " + prev);
        }
    }
}
```

Prac7

```
import hashlib, ecdsa

def wallet():
    k = ecdsa.SigningKey.generate(curve=ecdsa.SECP256k1)
    return k, k.get_verifying_key()

def transaction(priv, pub, msg):
    h = hashlib.sha256(msg.encode()).hexdigest()
    s = priv.sign(h.encode())
```

```
    print("Hash:", h, "\nSignature:", s.hex(), "\nValid:",
pub.verify(s, h.encode()))

a, ap = wallet()
b, bp = wallet()
transaction(a, ap, "Alice sends 0.5 BTC to Bob")
```

Prac8

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {
    string private data;

    function store(string memory s) public {
        data = s;
    }

    function retrieve() public view returns (string
memory) {
        return data;
    }
}
```